Decision Tree Classifiers:
An Analysis of Model Complexity and Overfitting

Module: Machine Learning

Assignment: Individual Machine Learning Tutorial

Student Name: **Beemanaboina Kranthikar**

Student ID: **24156005**

Github Repo: https://github.com/Kranthikar-123/Individual-Machine-Learning-Tutorial.git

1. Introduction

Machine learning is a fundamental part of modern data analysis that allows systems to learn from the data and make predictions on it automatically. Decision Trees are one of the most popular, simple and efficient Machine Learning approaches with great interpretability among all dimensions of machine learning techniques.

Decision Tree: Data is split into sub-data on the basis of feature value in DT, which forms a tree-like structure. Internal nodes correspond to a feature-based decision, and branches correspond to the outcome of that decision. Predictions are made at the leaf nodes and a class label is assigned. This structure makes Decision Trees quite intuitive and easy to visualise unlike many other machine learning models.

In this report, we will investigate the performance of Decision Tree classifiers and how it is affected by model complexity (as governed by tree depth). From experience, with increasing model complexity accuracy on training data improves while generalisation on unseen data can be terrible and is known as overfitting. On the other hand, models that are too simple might miss out on underlying structures which can lead to underfitting.

In a series of experiments and visualisations including accuracy curves, decision boundaries, feature importance analysis this study seeks to demonstrate the behavioural trade-offs in choosing an appropriate tree depth. The insight gained from findings helps to tune Decision Trees for better performance.

2. BACKGROUND / THEORY

Decision Trees is a supervised learning algorithm used for both classification and regression tasks. They work by recursively splitting the dataset into subsets based on feature values, creating a tree structure of decisions. This process aims to divide the data such that the resulting classes are as pure as possible with regards to the target variable.

At every node of the tree, the algorithm chooses one feature and a threshold that best divide the data. During the construction of a decision tree, this is done based on some splitting criterion which are generally one of the Gini impurity or entropy. Gini impurity: In a node, it measures the probability of classifying a randomly chosen sample incorrectly if it was labelled according to the distribution of classes in that node. A lower Gini value implies that the node is purer and that most samples belong to a single class. The goal of the algorithm is to minimize this impurity at each node.

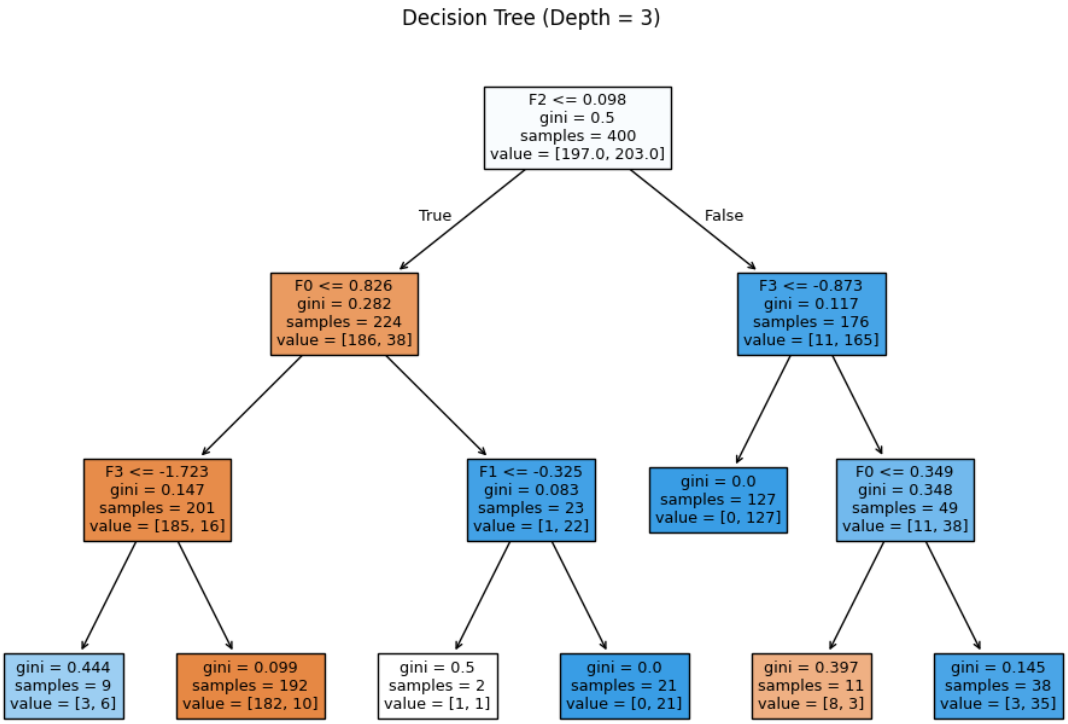
A Decision Tree is basically graphical and consists of three key components: the root node, internal nodes, and leaf nodes. The top node represents the whole population and is called the root node, while tree branches represent decision rules and leaf nodes (or terminal nodes) represent class labels. The terminal nodes, called leaf nodes, contain the final prediction: which class label to assign by taking a majority vote over the samples contained in that node.

Decision Trees are, one of the most interpretable machine learning model available. Building an incredibly complex function and we don’t actually know what is going on there unlike many other machine learning models, the decision-making process can be easily visualised and understood making them particularly useful in application where transparency is important.

But Decision Trees are very sensitive to the complexity of the model, specifically to how 'deep/wide' we can make a tree. The deeper the tree, the wider the depth of splits that can be made, which is controlled by the number of levels in a tree. Conversely, a shallow tree that doesn't reach deep enough into the dataset may fail to account for the underlying structure of the data and ultimately underfit it. What you described here is an underfitting model and as the name suggests it itself, the model in this case is too simple to capture the true relationship between target and features so has a bad performance on both train and test data.

Conversely, a tree that was too deep could create complex decision boundaries just for the sake of fitting the training data to perfection thereby compromising its generalisation capability. This is called overfitting. Overfitting happens when the model learns both real signals present in the data, and noise as well as random fluctuations.

Figure 1: Decision Tree structure showing feature-based splits, Gini impurity, and class predictions.



It is important to solve this trade-off underfitting and overfitting to create a good model. In this way, by tuning parameters like tree depth one can eventually reach a model that does an adequate job generalising while still retaining the relevant structure in the data. This dynamic equilibrium is a key aspect of the experiments described in this report.

3. METHODOLOGY

In this research, the behaviour of Decision Tree classifiers is studied using a synthetic dataset that is generated for controlled experimentation. We have generated a labelled dataset using make_classification function available in Scikit-learn library which generates labelled data with specified key characteristics. Five features were generated, resulting in 500 samples. Out of those a subset of features have been marked informative, participating significantly in classifying; others introduce redundant or noise. This arrangement allows a realistic assessment of the model's pattern recognition skill.

The dataset was divided into test and training subsets with a split of 80–20. The Decision Tree models were trained on the training set and the testing set was used to assess how well each model performed with unseen data. This separation ensures that the evaluation measures the generalising ability of that model, rather than just its ability to memorise the training data.

We used from the Scikit-learn DecisionTreeClassifier class to implement a Decision Tree classifier. The main parameter in this study is the max tree depth, which regulates the complexity of the model. To examine how this parameter affected performance, many models were fitted over a range of depths between shallow trees

(e.g. depth = 1) to deep trees (e.g. depth = 14). This systematic variation helps a clean comparison of how the model complexity affects performance.

To evaluate the performance of each model, classification accuracy was measured as the fraction of correctly predicted labels. To check for underfitting or overfitting, accuracy was calculated both for the training dataset and the testing data set. Alongside numeric evaluation, a number of visualisations were also produced to further understand model behaviour. Representations such as accuracy against tree depth plots, visualizations of the tree data structure, two-dimensional feature space decision boundary plots, feature importance metrics analysis and confusion matrices are presented.

It establishes an adequate methodology to mean pair-wise difference between models, providing insight on the effect of model complexity using quantitative metrics and visual analysis. This can provide a fine-grained view of the trade-offs that are being made in selecting models and specifying parameters.

4. IMPLEMENTATION

In Python, we implemented the Decision Tree model using the Scikit learn library. It started with creating a dummy classification dataset, which we then split into training and testing data to ensure fair evaluation of model performance.

We then instantiated a Decision Tree classifier using the DecisionTreeClassifier class. The implementation primarily focused on analysing the effect of the max_depth parameter, which determines the number of levels in arboreal outputs and thus, model complexity. Using different values for depth to train multiple models to see how performance is affected as the model become more complex.

At each depth value, the model was fitted against the training dataset and tested on both the train and test datasets. The accuracy scores were then save and kept for the comparisons. That enabled the generation of plots showing how model performance changes as depth increases. These comparisons are crucial for determining when the model starts to overfit.

As a next step to improve interpretability, the structure of the Decision Tree is visualized on the basis of plot_tree function. This visualisation shows the hierarchal decision making process in a model including the feature which was used, what were the threshold values and where samples were distributed across nodes. This also aids in interpreting how the model makes its predictions.

The dataset was then reduced to two features for a decision boundary plot. A grid mesh was established over the feature space; for each point on the grid it used out-trained model to predict class labels. Hence, it gave a graphical illustration of how Decision Tree separates the input regions.

The importance for different features was then analyzed based on the feature importance value extracted from the trained model. The values your output generates represent how much each input helped make the decision (relative to others). A confusion matrix was also generated to give a more detailed overview of the classification performance, clearly showing how many correct and incorrect classifications have been made for each class.

In sum, the implementation combines numerical assessment and graphical analysis for a thorough interpretation of Decision Tree functionalities.

5. RESULTS

Quantitative metrics and visual analysis were employed to evaluate the effectiveness of the Decision Tree classifier. The purpose was to see how changing the depth of the tree affects training/error accuracy and also a general behaviour of a model.

Figure 2: Training and testing accuracy as a function of tree depth.

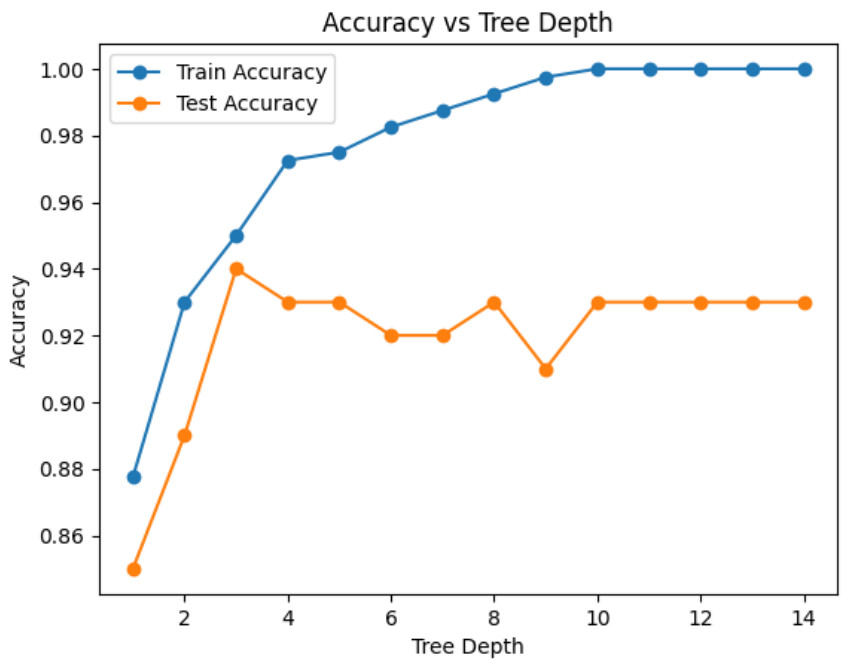
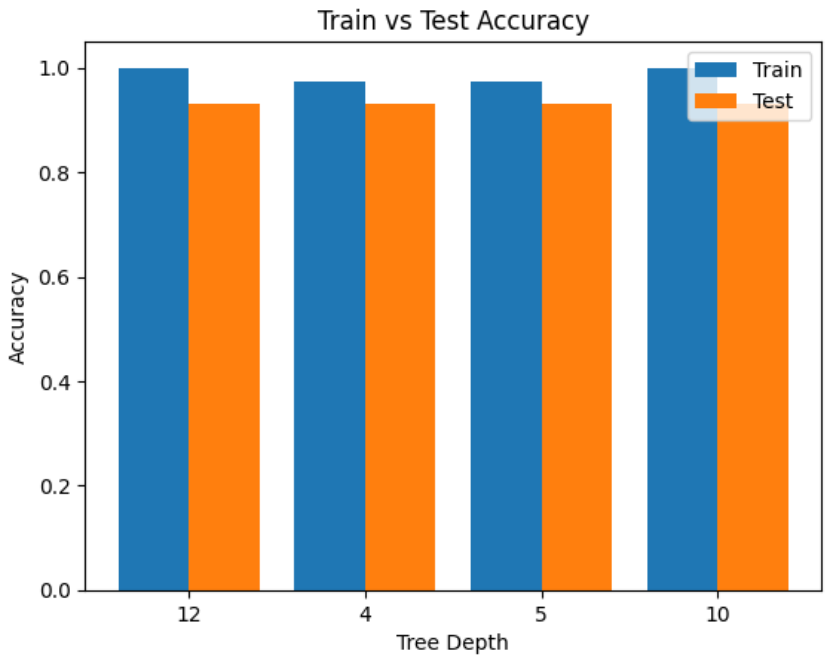


Figure 2 shows the relationship between tree depth and model accuracy, with training and testing accuracy over executive depth. In hours with low depth, the model has an underfitting with a poor level of accuracy for both training and testing datasets. These accuracies continue to improve with an increase in depth, but

the testing accuracy peaks at an intermediate depth. After this point, the training accuracy keeps increasing to nearly perfect performance and the testing accuracy starts going down. The main reason for the divergence is clearly overfitting happening.

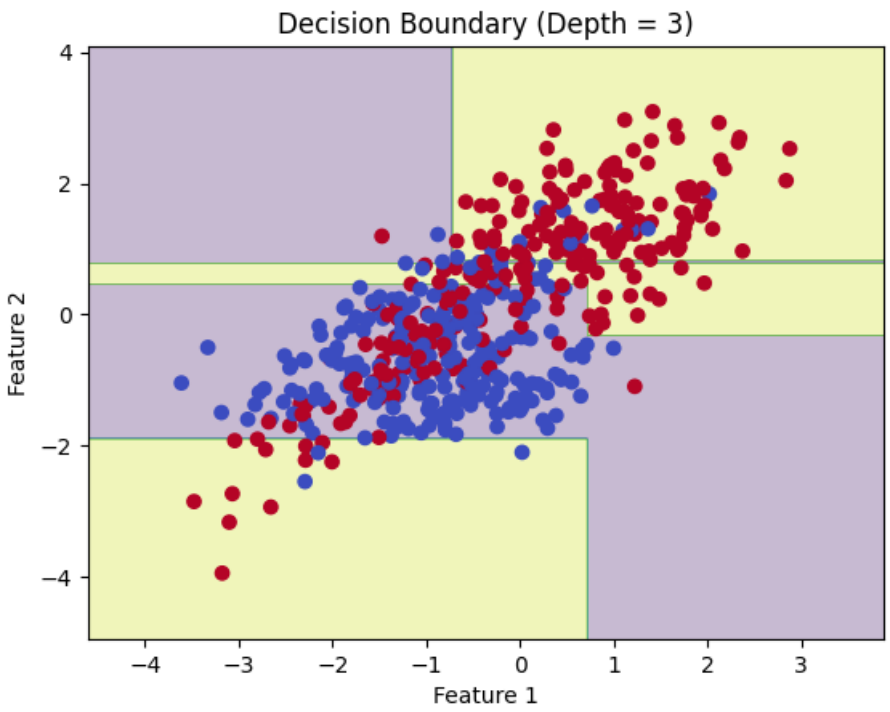
Figure 3: Comparison of training and testing accuracy across selected tree depths.



To drive this behaviour home, a bar chart is shown in Figure 3 which compares the training and testing accuracy achieved for selected tree depths. As the depth increases, the gap between training and test performance grows largerstill, reaffirming that deeper trees often learn by memorising the data rather thangeneralising.

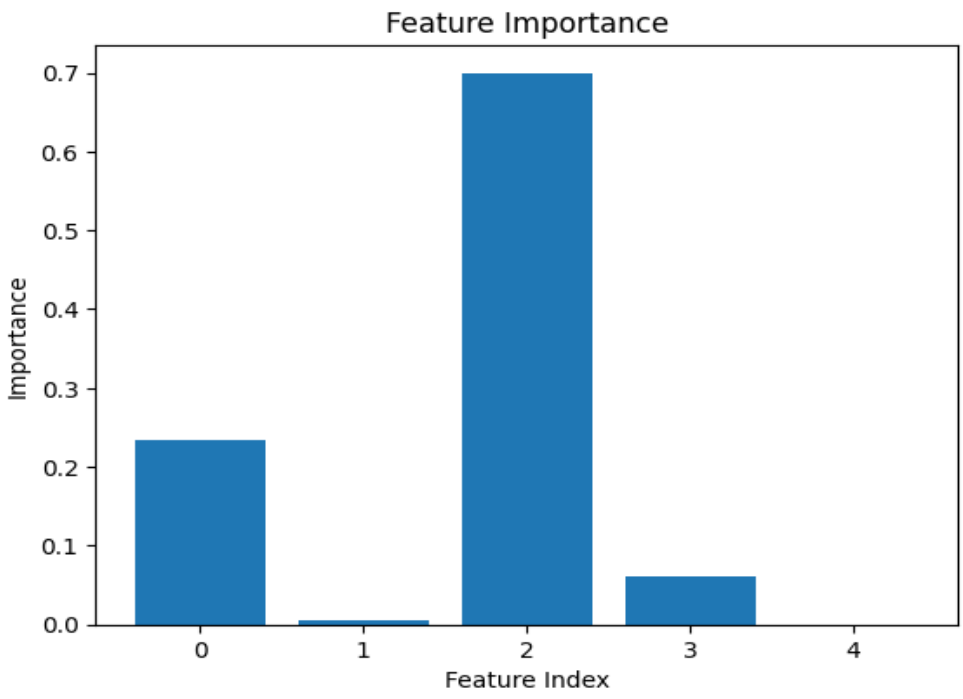
Figure 1 visualizes the structure of Decision Tree and shows how the model makes decisions. Each node in the tree corresponds to a feature and threshold which splits the data, whereas leaf nodes represent final class predictions. This is the hierarchical structure showing how model keeps on splitting up data recursively to improve classification accuracy.

Figure 4: Decision boundary illustrating how the Decision Tree partitions the feature space into axis-aligned regions.



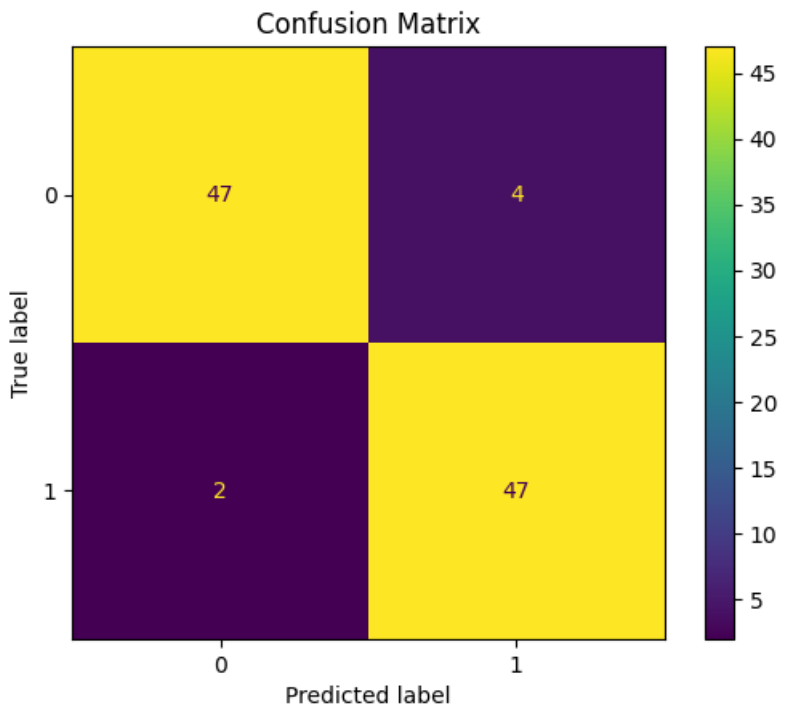
In Figure 4, the model's geometric interpretation is shown where the decision boundary is present in a two-dimensional feature space. The Decision Tree essentially splits the space into axis aligned rectangular areas as seen from the plot. In this method, regions are the decision rules learned in training, which show when a class label(additive)/ must be assigned with respect to feature thresholds.

Figure 5: Feature importance indicating the relative contribution of each feature to the model's predictions.



A feature importance analysis shown in Figure 5 provides additional information about the model's behaviour. This plot shows how much each feature contributes to the model's predictions. As you can see that some of the features are far more important and dominate outcome.

Figure 6: Confusion matrix showing classification performance on the test dataset.



Lastly, results concerning the classification performance are summarised in Figure 6, which shows the confusion matrix for test dataset. This tool gives us an idea of the number of correct and incorrect predictions made by the model for every class, allowing for a more comprehensive assessment than general accuracy

alone. The performance of the model seems excellent overall, but there are some misclassification that can be observed.

6. DISCUSSION

The results from the experiments are a convincing evidence for model complexity-trade off against generalisation in Decision Tree classifiers. Increasing the depth of the tree allows these models to learn more complex patterns in your training set. As evidenced by the consistent rise of training accuracy with increasing tree depth But this added complexity leads to a loss of generalisation.

And, at lower depths, the model is too simple to realize any underlying structure in the data resulting in underfitting. In this case, both accuracies during training and testing are low as the model does not learn anything meaningful. Performance improves on training and testing datasets as we make the model more expressive by increasing its depth.

However, after some depth the model starts to fit the training data too closely. This is why training accuracy keeps on growing while testing one decreases — the noticeable difference between training and testing accuracy. Overfitting happens because the model learns noise and small fluctuations in the training data instead of learning the actual signal.

This observation is also supported by the decision boundary visualisation. Given that models grow millions of tree leaves, (at lower depths these are computationally inefficient) but as depth increases the model recursively refines the partitioning of the feature space providing highly specific partitions of each tree leaf, which can overfit on unseen data. In a similar vein, the analysis conducted on feature importance reveals that in some cases certain features tend to prevail, and if a model places too much emphasis on specific characteristics, overfitting can occur.

Overall, these results highlight the need to control for model complexity through parameters such as tree depth. Choosing a suitable depth is important in order to reduce both bias and variance, allowing for better prediction performance.

7. CONCLUSION

This investigation delves into the characteristic behaviour of Decision Tree classifiers, in particular, how the depth of a model affects its performance. By making some experiments and conditions, it was shown that depth of trees is the fundamental function in balancing underfitting and overfitting. The study showed that a shallow tree could not capture meaningful patterns in the data while an overly deep tree would memorize the training data causing overfitting and poor generalization.

This showed that there is an ideal depth for which the model will output the best predictions on data it has not seen. Understanding Model: Tools like accuracy plots, decision boundaries and feature importance graphs were used to help understand how the model works and how complexity affects the behaviour features.

So, this was the discussion about Decision Trees where overall it is a strong classifier and interpretable but sensitive to changes in data preparation as well as requires an effort for hyperparameter tuning. To ensure that predictions are robust and reliable, proper model complexity must be set. This study further supports the need for careful balance between model flexibility and generalisation in machine learning applications.